ProofPay – Dev 3 Completion Report & Merge Guide

Author: Dev 3 | **Branch:** `dev3` | **Status:** ✅ **COMPLETE – ready for review and merge into `main`**

Scope: Synthetic Data Generator, Customer Simulator, Reconciliation Engine, Autonomous Recovery Agent, Evaluation Harness, and Dev 4 REST API.

> **Core Architectural Law: "The agent proposes, the policy decides, code executes."** Recovery code **never** imports PayPal or email transport code. It only builds immutable `Proposal` objects.

---

1. Summary at a Glance

Item	Result
Tests	**159 passing** (`pytest backend/tests -v`)
Synthetic customers	**120**
Seeded cases	**150** (90 failed subscriptions + 60 overdue invoices)
Transaction records	**271** multi-source records
Escrow projects	**10**
Planted traps	**54**, with **zero leaks** to the agent
Reconciliation accuracy	**100% precision / 100% recall**
Wrongful contacts (ProofPay)	**0** (100% avoided)
Wrongful contacts (Naive baseline)	**81**
Disputes avoided	**63**

---

2. What Is Done (The 5 Subsystems)

2.1 Synthetic Data Generator & Secret Answer Key

Location: `backend/app/simulator/datagen/`

- Deterministic generator (**seed = 42**) – same data every run.
- Produces 120 customers, 90 failed subscriptions, 60 overdue invoices, 271 transaction records and 10 escrow projects.
- Plants **54 traps** in `data/answer_key/traps.json`: bank-transfer payers, rounding gaps, fee deductions, disputes, fraud declines and over-limit release attempts.
- **Ground truth is strictly segregated.** AST tests prove the agent never imports from `data/answer_key/`.

2.2 Reconciliation Engine (3-Pass Matcher)

Location: `backend/app/reconciliation/`

- **Pass 1:** Exact ID match.
- **Pass 2:** Customer + date + tolerances (**$0.05 rounding**, fee deductions).
- **Pass 3:** Rapidfuzz text match.

Confidence	Tier	Outcome
≥ 0.90	`auto_match`	Case closed as `PAID_ELSEWHERE` with **0 customer contacts**
0.60 – 0.89	`human_review`	Case paused in `ACTION_PENDING_APPROVAL`; ops review task created
< 0.60	`no_match`	Case continues down the recovery ladder

Also includes escrow **pre-release** and **post-release** verification checks.

2.3 Autonomous Recovery Agent

Location: `backend/app/agents/recovery/agent.py` (+ `backend/app/recovery/`)

- Diagnoses failure causes: `insufficient_funds`, `expired_card`, `temporary_hold`, etc.
- Prioritizes by **expected value (Amount × Likelihood)** with a daily cap.
- Tailored recovery ladders with localized **Jinja2 templates in English and Hindi**.

- **Safety rules enforced:**
  - **Quiet hours:** no contact 21:00–08:00 in the customer's local time.
  - **48-hour minimum gap** between emails.
  - **Maximum 4 contacts** per case.
  - **$200 limit** – larger actions require ops approval.
  - **Immediate halt** on customer dispute or opt-out, cancelling all scheduled future jobs.

2.4 Benchmarking & Evaluation Harness

Location: `backend/app/eval/`

- Runs the ProofPay agent vs a **Naive Baseline** across all 150 seeded cases.
- Scorecard: **0 wrongful contacts vs 81** for the baseline, and **63 disputes avoided**.
- Results served via `GET /eval/latest`.

2.5 Dev 4 REST API

Location: `backend/app/api/` (thin FastAPI routers)

Endpoint	Purpose
`GET /cases?sort=priority`	Prioritized recovery queue
`GET /cases?status=HALTED,UNRECOVERED`	Filtered cases; mandatory `reason` on every row
`GET /cases/{id}`	Case detail, attempt timeline, safety policy audit trail
`GET /outbox`	Messages sent to the customer inbox
`GET /reconciliation/summary`	`matched`, `review`, `exception` metrics
`GET /approvals`	Ops approval queue (>$200 actions)
`POST /approvals/{id}/decide`	Approve / reject an action
`GET /dashboard/headline`	`recovered_amount`, `at_risk_amount`, `case_count`, `recovered_count`
`GET /eval/latest`	Benchmark scoreboard

Endpoint	Purpose
`GET /fixtures`	Escrow dispute evidence fixtures
`POST /demo/{inject-failure, run-simulator, advance-clock, reset}`	Simulation controls – **only when** **`DEMO_MODE=True`**, otherwise `404`

---

3. Workflow: How a Case Moves Through the System

1. **Trigger arrives** – `subscription.payment_failed`, `invoice` overdue, or `funding.failed` / `instalment.failed`.
2. **`process_case()`** ingests the trigger and **dedupes** it.
3. **Reconciliation check** – `check_already_paid()` asks: *"Has this person already paid, in any source?"*
   - `auto_match` → close as `PAID_ELSEWHERE`, **stop. No contact.**
   - `human_review` → pause in `ACTION_PENDING_APPROVAL`.
   - `no_match` → continue.
4. **Diagnose** failure cause (LLM budget capped at **2 calls**).
5. **Prioritize** by expected value with a daily cap.
6. **Strategy** selects the next rung of the recovery ladder; **timing** applies quiet hours, 48h gap and payday alignment.
7. **Agent builds a** **`Proposal`** (immutable). It never sends anything itself.
8. **Policy** (`policy.check(proposal)`) returns a `PolicyDecision` / `Verdict`.
9. **Executor** (`executor.execute(proposal, decision)`) performs the action with idempotency and dispatches to the outbox.
10. **State machine** records an audited transition. Terminal states: `RECOVERED`, `HALTED`, `UNRECOVERED`, `PAID_ELSEWHERE`.
11. **Bridges** publish escrow events (`funding.resumed`, `project.flagged_for_freelancer`) on state change.

---

4. Directory Structure

```
backend/app/
├── agents/recovery/agent.py      # Core Recovery Agent orchestrator
├── simulator/
│   ├── datagen/                  # Synthetic data & trap generation
│   ├── profiles.py               # Customer simulator behavior engine
│   └── sim_events.py             # Simulator event dispatcher
├── reconciliation/
│   ├── normalizer.py             # Normalized ReconRecord parser
│   ├── matcher.py                # 3-pass reconciliation matcher
│   ├── tiers.py                  # Confidence tiers
│   ├── metrics.py                # Precision / recall metrics
│   └── escrow_recon.py           # Pre/post-release escrow checks
├── recovery/
│   ├── state_machine.py          # Lifecycle states & audited transitions
│   ├── diagnose.py               # Failure diagnosis & LLM call budget
│   ├── prioritize.py             # Expected value prioritization
│   ├── strategy.py               # Recovery ladders & action selection
│   ├── timing.py                 # Quiet hours, 48h gap, payday alignment
│   ├── tools.py                  # Proposal factory tools
│   ├── bridges.py                # Escrow event bridges
│   ├── constants.py              # Event name constants
│   └── templates/                # Jinja2 localized templates (en / hi)
├── channels/                     # Outbox & mock email adapter
├── eval/                         # Scoreboard runner & baseline
├── api/                          # FastAPI routers for Dev 4
└── stubs/                        # TEMPORARY teammate stand-ins
data/
├── generated/                    # Public seeded JSON & CSV
└── answer_key/                   # SECRET ground truth (profiles & traps)
fixtures/                         # Escrow dispute evidence fixtures
README_DEV3.md                    # Handover guide (repo root)
```

---

5. ⚠️ Things to Consider BEFORE Merging into `main`

5.1 Temporary stubs must be replaced

All stand-ins live in `backend/app/stubs/`. Each file starts with `# TEMP: replaced by <real folder>`.

Stub file	Replaced by	Exports	Owner
`contracts_stub.py`	`contracts/`	`Proposal`, `PolicyDecision`, `Verdict`, `ActionKind`, `CaseState`, `FINAL_STATES`	**Dev 1**
`clock.py`	`core/clock/`	`utcnow()`, `set_time()`, `advance_time()`, `simulate_days()`, `reset()`, `register_tick_listener()`	**Dev 1**
`audit.py`	`audit/`	`log_event(event_type, payload)`, `get_events()`, `clear()`	**Dev 1**
`repositories.py`	`db/`	`case_repo` (`create`, `get`, `update`, `list_all`, `reset`), `contact_repo` (`add`, `list_for_case`)	**Dev 1**
`scheduler.py`	`scheduler/`	`enqueue(func, *args, **kwargs)`, `cancel_for_case(case_id)`, `remove_job(id)`	**Dev 1**
`policy.py`	`policy/`	`check(proposal) -> PolicyDecision` (48h gap, quiet hours, cap, $200 limit)	**Dev 2**
`executor.py`	`executor/`	`execute(proposal, decision) -> receipt`	**Dev 2**
`llm.py`	`llm/`	`call(prompt, system_prompt, schema, case_id) -> dict` (max 2 budget)	**Dev 2**

To replace a stub: change the imports in `backend/app/stubs/` to re-export the real module (or update the import sites to the production package), run the full test suite, then delete the stub file.

5.2 Merge invariants – **DO NOT BREAK**

1. **Never** allow recovery or reconciliation code to import payment (PayPal) or email drivers. Communicate **only** through `Proposal` objects.
2. **Never** allow recovery code to import from `data/answer_key/`. Only `simulator/`, `reconciliation/metrics.py` and `eval/` may.
3. **Terminal states are immutable.** A case in `RECOVERED`, `HALTED`, `UNRECOVERED` or `PAID_ELSEWHERE` must never transition except through an audited human `reopen()`.
4. **Zero wrongful contacts:** every already-paid customer must receive exactly **0** contacts.

5.3 Event names to confirm with the gateway & escrow teams

Defined in `backend/app/recovery/constants.py`:

Constant	Event string	Publisher	Effect
`EVENT_FUNDING_FAILED`	`funding.failed`	PayPal / Escrow webhook	Opens `escrow_funding` case
`EVENT_INSTALMENT_FAILED`	`instalment.failed`	Escrow milestone webhook	Opens `escrow_funding` case with milestone metadata
`EVENT_FUNDING_RESUMED`	`funding.resumed`	ProofPay Recovery Agent	Published when funding case reaches `RECOVERED`
`EVENT_PROJECT_FLAGGED`	`project.flagged_for_freelancer`	ProofPay Recovery Agent	Published when funding case reaches `UNRECOVERED`
`EVENT_INVOICE_PAID`	`invoice.paid`	PayPal webhook	Closes case as `RECOVERED`
`EVENT_INVOICE_CANCELLED`	`invoice.cancelled`	PayPal webhook	Closes case as `UNRECOVERED`
`EVENT_SUBSCRIPTION_FAILED`	`subscription.payment_failed`	PayPal webhook	Triggers recovery case

5.4 Makefile targets Dev 1 must add

```
seed:
	python scripts/seed.py

eval:
	python backend/scripts/run_eval.py

demo-reset:
	python backend/scripts/demo_reset.py

test-dev3:
	python -m pytest backend/tests -v
```

> **Verify before merging:** `seed` points to `scripts/seed.py` while the others live under `backend/scripts/`. Confirm the real path of the seed script and fix the Makefile if needed.

5.5 Other checks

- **`DEMO_MODE`** must be `False` in production, otherwise `/demo/*` is exposed (it returns `404` when disabled).
- **`data/answer_key/` is secret ground truth** – make sure it is never packaged into a production image or exposed by the API.
- **Windows paths:** the project was developed at `d:\paypal\`. Check scripts for hard-coded backslashes or drive letters before running on Linux/CI.
- **Terminology:** DB-layer replacement (`db/`) must keep the `case_repo` / `contact_repo` method names, or the recovery code will break.

---

6. Pre-Merge Checklist

- [ ] `git status` clean on `dev3`; everything pushed to `origin/dev3`
- [ ] `pytest backend/tests -v` → **159 passed**
- [ ] `README_DEV3.md` read by Dev 1, Dev 2 and Dev 4
- [ ] Dev 1 confirmed: contracts, clock, audit, db, scheduler ready
- [ ] Dev 2 confirmed: policy, executor, llm ready
- [ ] Event names in §5.3 confirmed with gateway/escrow owners
- [ ] Makefile targets added and `seed` path verified
- [ ] `.importlinter` contracts pass (no recovery → PayPal/channels imports)
- [ ] `test_datagen_ast.py` passes (answer key privacy)
- [ ] `test_ig3.py` passes (zero wrongful contacts)
- [ ] `make eval` still reports **0 vs 81** after stub replacement

---

7. How to Merge `dev3` into `main`

Recommended: Pull Request (safest).

```
# 1. Make sure dev3 is up to date and pushed
git checkout dev3
git pull origin dev3
git push origin dev3

# 2. Bring the latest main INTO dev3 first and resolve conflicts here (not on
main)
git fetch origin
git merge origin/main
#   → resolve conflicts (most likely: Makefile, requirements, app/main.py
router registration)

# 3. Re-run the full test suite and eval
pytest backend/tests -v
make eval

# 4. Push the merged branch
git push origin dev3

# 5. Open a Pull Request: dev3 → main on GitHub/GitLab
#    Request review from Dev 1, Dev 2 and Dev 4. Merge after approval + green
CI.
```

Alternative: local merge (only if no PR process).

```
git checkout main
git pull origin main
git merge --no-ff dev3 -m "Merge Dev 3: reconciliation, recovery agent, eval,
API"
pytest backend/tests -v
git push origin main
```

Likely conflict hot-spots:

- `Makefile` – combine all targets, do not overwrite.
- `requirements.txt` / `pyproject.toml` – keep **both** sides (Dev 3 adds `rapidfuzz`, `jinja2`, `fastapi`, `pytest`).
- `backend/app/main.py` – register the Dev 3 routers alongside existing ones.
- `.importlinter` – merge contracts, do not drop any.

---

8. After Merging: Stub Replacement Steps

1. Dev 1 merges real `contracts/`, `core/clock/`, `audit/`, `db/`, `scheduler/`.
2. Dev 2 merges real `policy/`, `executor/`, `llm/`.
3. Update imports in `backend/app/stubs/` to point to the real modules (one stub at a time).
4. Run `pytest backend/tests -v` after **each** replacement.
5. When all are replaced, delete `backend/app/stubs/`.
6. Run `make seed`, `make eval`, `make demo-reset` and confirm the scoreboard is unchanged.

---

9. Public Functions Other Devs Should Call

Function	Module	What it does
`process_case(case_id=None, trigger=None) -> dict`	`backend.app.agents.recovery.agent`	**Main entry point.** Dedupes, reconciles, diagnoses, prioritizes, builds proposal, evaluates policy
`halt_case(case_id, reason="manual_halt", actor="operator") -> dict`	`backend.app.recovery.state_machine`	Halts case and **cancels all scheduled jobs**. Idempotent
`reopen(case, actor="human", reason=...) -> dict`	`backend.app.recovery.state_machine`	Audited human override of `HALTED` / `UNRECOVERED` back to `OPEN`. **Rejects non-human actors**
`check_already_paid(case) -> str`	`backend.app.reconciliation.matcher`	Returns `auto_match`, `human_review` or `no_match`
`pre_release_check(milestone) -> bool`	`backend.app.reconciliation.escrow_recon`	**Blocks** escrow release if funding capture is missing
`post_release_check(milestone) -> bool`	`backend.app.reconciliation.escrow_recon`	Verifies payout appears in records after release
`handle_funding_failed(project_id, client_id, amount_usd, failure_code)`	`backend.app.recovery.bridges`	Opens `escrow_funding` case
`handle_instalment_failed(project_id, milestone_id, client_id, amount_usd, failure_code)`	`backend.app.recovery.bridges`	Opens `escrow_funding` case for a milestone
`on_escrow_case_state_change(case) -> str | None`	`backend.app.recovery.bridges`	Publishes `funding.resumed` / `project.flagged_for_freelancer`

---

10. Instructions Per Teammate

Dev 1 (core infra): Replace stubs for contracts, clock, audit, db, scheduler. Add the Makefile targets. Keep `CaseState` terminal states immutable. Keep `case_repo` / `contact_repo` method names.

Dev 2 (policy / executor / LLM): Replace `policy`, `executor`, `llm` stubs. Policy must enforce 48h gap, quiet hours, daily cap and the **$200 limit**. Executor must be **idempotent**. LLM tracker must enforce the **max-2 calls per case** budget.

Dev 4 (frontend / dashboard): Consume the endpoints in §2.5. Every `HALTED` / `UNRECOVERED` row carries a mandatory `reason`. `/demo/*` only works when `DEMO_MODE=True`.

---

11. How to Run & Verify

```
make seed         # generate synthetic data + answer key
make eval         # ProofPay vs Naive Baseline scoreboard
make demo-reset   # full deterministic wipe + reseed (<30s)
make test-dev3    # run the full Dev 3 test suite (159 tests)
```

Expected results: 159 tests pass · 0 wrongful contacts (vs 81 baseline) · 63 disputes avoided · 100% reconciliation precision/recall.

---

12. Architectural Invariant Checks (Automated)

- **Import boundary** – `.importlinter` + `test_import_linter.py`: recovery/reconciliation never import PayPal or channels.
- **Answer-key privacy** – `test_datagen_ast.py` (AST scan): only `simulator/`, `reconciliation/metrics.py` and `eval/` import `data/answer_key/`.
- **Zero wrongful contacts** – `test_ig3.py`: already-paid customers receive exactly 0 contacts.

> **Rule of thumb for reviewers:** if any of these three tests fails after a merge, **do not merge to `main`**.

---

13. Team Announcement (Ready to Post)

> Hey team! 👋 Dev 3 part is complete and pushed to branch `dev3`. All **159 tests** are passing (`pytest backend/tests -v`).
>
> Built: synthetic data + secret answer key (120 customers, 150 cases, 271 records, 54 traps, zero leaks) · 3-pass reconciliation engine (100% precision/recall) · autonomous recovery agent (quiet hours, 48h gaps, $200 limit, halts on disputes) · eval harness (**0 wrongful contacts vs 81** for baseline) · FastAPI endpoints for Dev 4.
>
> 📄 Please read `README_DEV3.md` in the repo root before merging. It lists all temporary stubs (`backend/app/stubs/`), callable functions, event names and Makefile targets.